

★★★ 반드시 내 것으로 ★★★

#MUSTHAVE

실무 중심 이론, 실무 중심 프로젝트로 단단하게 시작하자

성낙현의 JSP 자바 웹 프로그래밍



우리는
가치가 성장하는
시간을
만듭니다.



[1단계] 빠르게 익히는 JSP 기초

00 개발 환경 구축

- 01 JSP 기본
- 02 내장 객체(Implicit Object)
- 03 내장 객체의 영역(Scope)
- 04 쿠키(Cookie)
- 05 데이터베이스
- 06 세션(Session)
- 07 액션 태그(Action Tag)
- [Project] 08 모델1 방식의 회원제 게시판 만들기
- [Project] 09 게시판에 페이징 기능 넣기

[2단계] 고급 기능으로 스킬 레벨업

- 10 표현 언어(EL : Expression Language)
- 11 JSP 표준 태그 라이브러리(JSTL)
- 12 파일 업로드 및 다운로드
- 13 서블릿(Servlet)
- [Project] 14 모델2 방식(MVC 패턴)의 자료실형 게시판 만들기

[3단계] 프로젝트로 익히는 현업 스킬

- [Project] 15 웹소켓으로 채팅 프로그램 만들기
- [Project] 16 SMTP를 활용한 이메일 전송하기
- [Project] 17 네이버 검색 API를 활용한 검색 결과 출력하기
- [Project] 18 배포하기

Chapter

01

JSP 기본

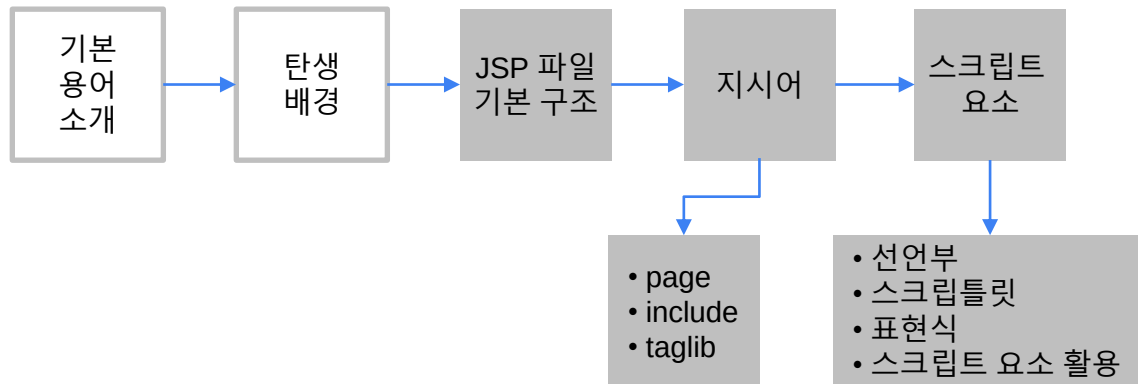


□ 학습 목표

□ 학습 순서

□ JSP란

JSP의 개념, 탄생 배경, 동작 원리의 이해, JSP 파일의 기본 구조와 핵심 요소 학습



- JSP는 동적인 웹 페이지를 개발하기 위한 웹 프로그래밍 기술
- 자바 언어로 서버 측에서 웹 페이지들을 생성해 웹 브라우저로 전송

□ 장점

- 짧은 코드로 동적인 웹 페이지를 생성
- 기본적인 예외는 자동으로 처리
- 많은 확장 라이브러리를 사용
- 스레드 기반으로 실행되어 시스템 자원을 절약

□ 활용 사례

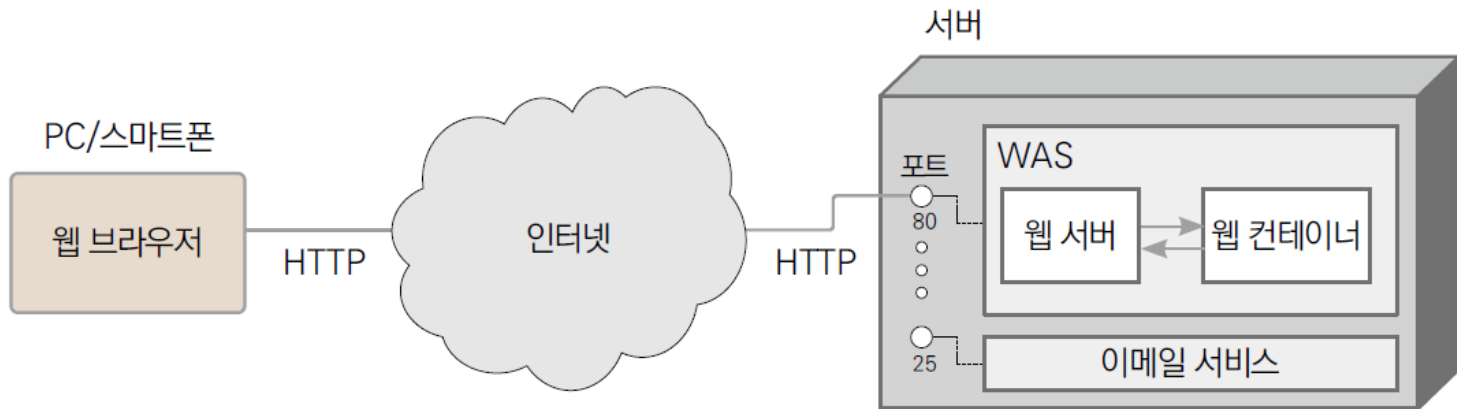
- JSP는 기업용 자바 기술의 집합체인 Java EE(Java Platform, Enterprise Edition)의 핵심 요소
- Java EE는 대한민국 정부 표준 프레임워크의 근간
- 정부나 공기업 주도의 사업 등 대규모 기업용 시스템 구축에 주로 사용
- 클라우드 시대가 되면서 구글 앱 엔진과 아마존 웹 서비스(AWS) 등에서도 지원하기 시작하면서 활용 폭이 더욱 넓어짐

JSP의 구성 요소

- 정적인 데이터 (텍스트, HTML 코드)
- 지시어 (디렉티브, Directive) 교재 1장
- 스크립트 (Script): 교재 1장
 - ◎ 표현식(Expression), 스크립트릿(Scriptlet), 선언부(Declaration)
- 기본 객체 (implicit object) 교재 2장
- 액션 태그 (Action Tag) 교재 7장
- 표현 언어 (Expression Language) 교재 10장
- 커스텀 태그 (Custom Tag)
- 주석
 - ◎ HTML주석, JSP주석, java주석

기본 용어 소개

- 서버(Server)
- 웹 서버(Web Server)
- 웹 컨테이너(Web Container)
- WAS(Web Application Server)
- HTTP(HyperText Transfer Protocol)
- HTTPS(HTTP Secure)
- 프로토콜(Protocol)
- 포트(Port)



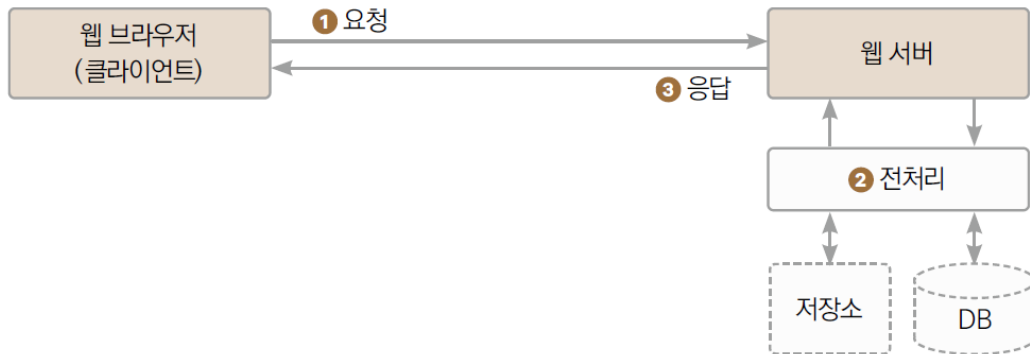
1.1 동적 웹 페이지로의 여정과 JSP(1)

1.1.1 정적 웹 페이지와 동적 웹 페이지

- 정적 웹 페이지 구동 방식



- 동적 웹 페이지 구동 방식



1.1 동적 웹 페이지로의 여정과 JSP(2)

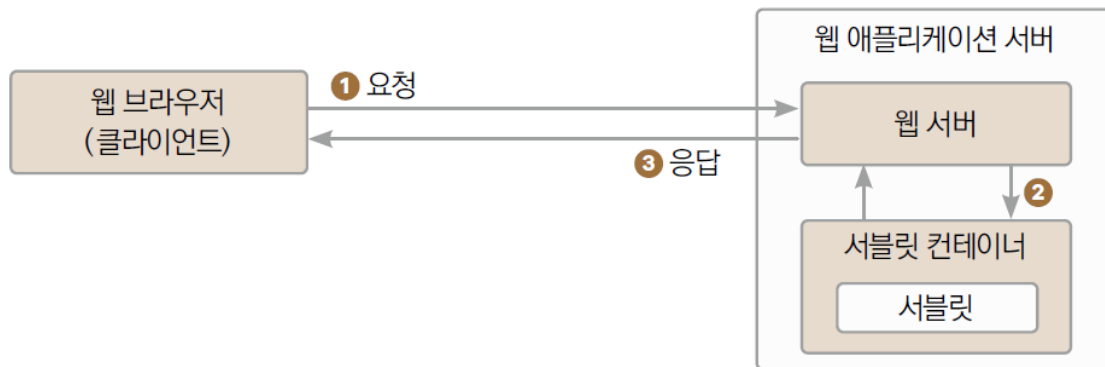
1.1.2 애플릿, 동적 웹을 향한 자바의 첫걸음

- 동적 웹 페이지 기술로 분류되지는 않지만 웹을 동적으로 만들기 위한 고대의 자바 기술이 바로 자바 애플릿
- 자바 애플릿은 웹에서 실행되도록 설계된 자바 애플리케이션을 통째로 웹 브라우저로 전송한 후, 자바 가상 머신을 탑재한 웹 브라우저가 이를 실행하는 방식으로 구동
- 동적 웹 기술이 발달하기 전 시절에는 한때 이목을 끌었지만, 표준 기술인 HTML과 자바스크립트가 발전하면서 지금은 더 이상 지원되지 않는 추억의 기술

1.1 동적 웹 페이지로의 여정과 JSP(3)

1.1.3 서블릿, 자바 웹 기술의 새 지평을 열다

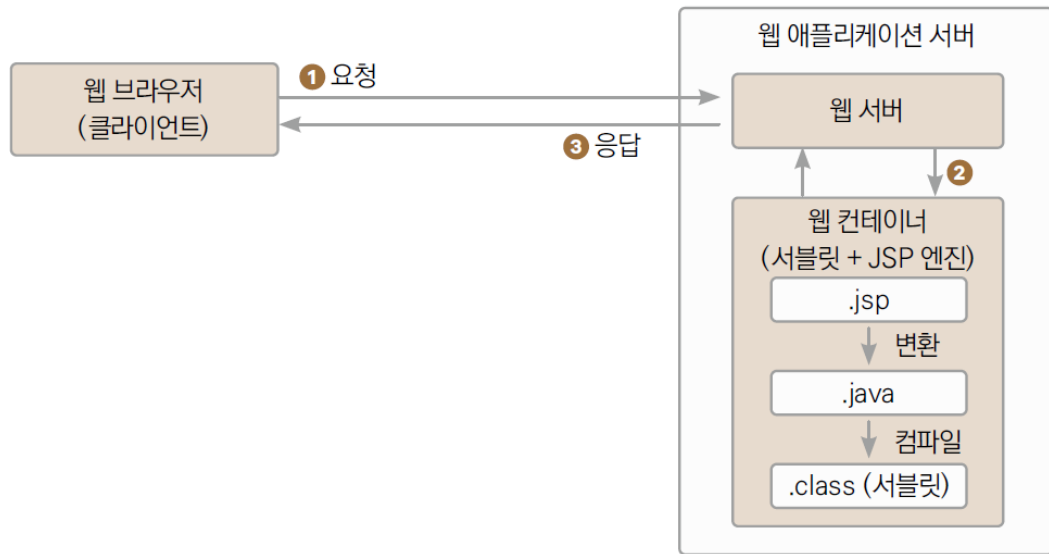
- 자바 애플릿: 애플리케이션 전체가 클라이언트에 다운로드된 후 실행. 속도, 보안, 유연성 등에서 한계
- 서블릿: 서버 측에서 실행. 클라이언트의 요청을 받으면 서버에서 처리한 후, 응답으로는 결과값만 보내주는 구조
- 자바 파일(.java)을 컴파일한 클래스 파일(.class) 형태. 실행하고 관리해주는 런타임이 서블릿 컨테이너이며. 아파치 톰캣이 대표적인 서블릿 컨테이너



1.1 동적 웹 페이지로의 여정과 JSP(4)

1.1.4 JSP, 자바 웹 기술의 최종 진화

- 기본을 HTML로 하고 필요한 부분만 자바 코드를 삽입하는 형태인 JSP
- JSP는 클라이언트에 보여지는 결과 페이지를 생성할 때 주로 쓰이며, 서블릿은 UI 요소가 없는 제어나 기타 처리 용도로 사용



1.1 동적 웹 페이지로의 여정과 JSP(5)

1.1.4 JSP, 자바 웹 기술의 최종 진화

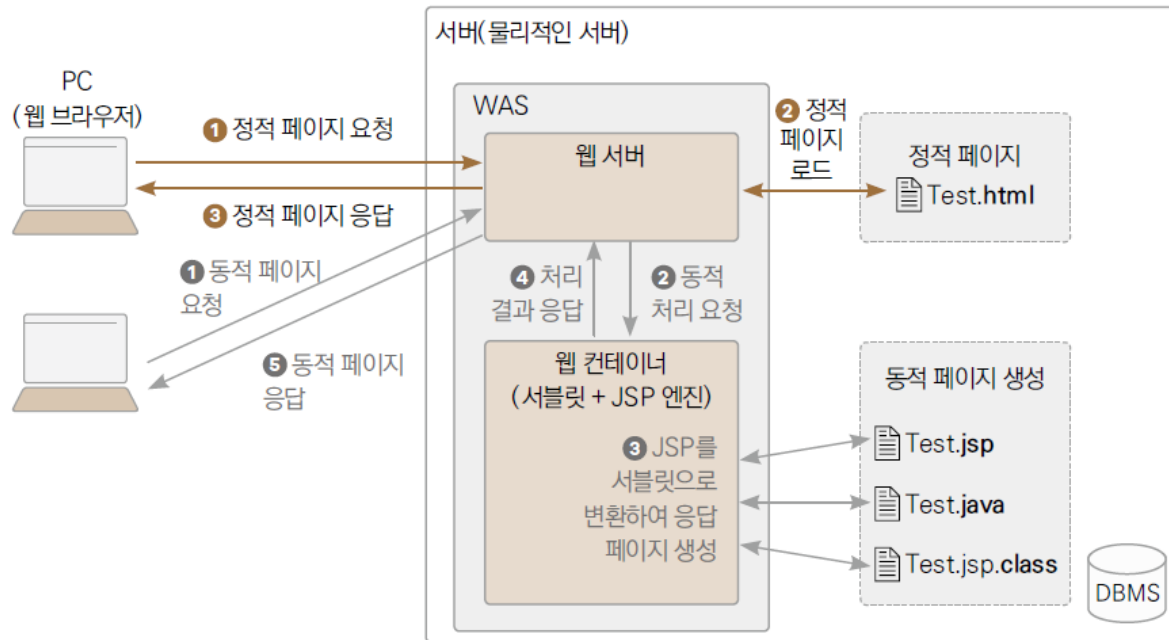
- 서블릿과 JSP 기술의 차이점

서블릿	JSP
자바 코드 안에서 전체 HTML 페이지를 생성	HTML 코드 안에서 필요한 부분만 자바 코드를 스크립트 형태로 추가
변수 선언 및 초기화가 반드시 선행되어야 함	자주 쓰이는 기능을 내장 객체로 제공하여 즉시 사용 가능
컨트롤러(Controller)를 만들 때 사용	처리된 결과를 보여주는 뷰(View)를 만들 때 사용

1.1 동적 웹 페이지로의 여정과 JSP(6)

1.1.5 오늘날의 웹 사이트

- 오늘날의 일반적인 웹 구동 방식(자바 중심)



1.2 JSP 파일 기본 구조

- JSP의 주된 목적은 웹 브라우저에 띄울 HTML 파일을 생성

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
pageEncoding="UTF-8"%>
<%!
String str1 = "JSP";
String str2 = "안녕하세요..";
%>
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>HelloJSP</title>
</head>
<body>
    <h2>처음 만들어보는 <%= str1 %></h2>
    <p>
        <%
        out.println(str2 + str1 + "입니다. 열공합시다^^*");
        %>
    </p>
</body>
</html>
```

지시어

스크립트 요소(선언부)

스크립트 요소(표현식)

스크립트 요소
(스크립틀릿)

1.3 지시어(Directive)(1)

- 지시어(지시자, 디렉티브): JSP 페이지를 자바(서블릿) 코드로 변환하는 데 필요한 정보를 JSP 엔진에 알려주며, 주로 스크립트 언어나 인코딩 방식 등을 설정

```
<%@ 지시어종류 속성1="값1" 속성2="값2" ... %>
```

- page 지시어 : JSP 페이지에 대한 정보를 설정
- include 지시어 : 외부 파일을 현재 JSP 페이지에 포함시킴
- taglib 지시어 : 표현 언어에서 사용할 자바 클래스나 JSTL을 선언

1.3 지시어(Directive)(2)

1.3.1 page 지시어

속성	내용	기본값
Info	페이지에 대한 설명을 입력	없음
Language	페이지에서 사용할 스크립팅 언어를 지정	java
contentType	페이지에서 생성할 MIME 타입을 지정	없음
pageEncoding	charset과 같이 인코딩을 지정	ISO-8859-1
Import	페이지에서 사용할 자바 패키지과 클래스를 지정	없음
Session	세션 사용 여부를 지정	true
Buffer	출력 버퍼의 크기를 지정합니다. 버퍼를 사용하지 않으려면 "none"으로 지정	8KB
autoFlush	출력 버퍼가 모두 채워졌을 때 자동으로 비울 지를 결정. buffer 속성이 none일 때 false로 지정하면 에러가 발생	true
trimDirective Whitespaces	지시어 선언으로 인한 공백을 제거할지 여부를 지정	false
errorPage	해당 페이지에서 에러가 발생했을 때 에러 발생 여부를 보여줄 페이지를 지정	없음
isErrorPage	해당 페이지가 에러를 처리할지 여부를 지정	false

1.3 지시어(Directive)(3)

1.3.1 page 지시어

- language, contentType, pageEncoding 속성

```
<%@ page language="java" contentType="text/html; charset=UTF-8"  
    pageEncoding="UTF-8"%>
```

- language : 스크립팅 언어는 자바를 사용
- contentType : 문서의 타입, 즉 MIME 타입은 text/html이고, 캐릭터셋은 UTF-8
- pageEncoding : 소스 코드의 인코딩 방식은 UTF-8
- import 속성
 - java.lang 패키지에 속하지 않은 클래스를 JSP 문서에서 사용하기 위해 임포트

1.3 지시어(Directive)(4)

- `errorPage`, `isErrorPage` 속성
 - 실행 도중에 에러가 발생하면 "HTTP Status 500" 에러 화면을 웹 브라우저에 표시
 1. `try/catch`를 사용하여 직접 에러를 처리
 - 예외 발생 부분을 `try/catch` 구문으로 감싸기. 이 페이지는 실행하는 즉시 예외가 발생하므로 `catch`절이 실행
 2. `errorPage`, `isErrorPage` 속성을 사용하여 디자인이 적용된 페이지로 대체
 - 개발자가 지정한 JSP 화면을 보여줌
 - ① 우선 `ErrorPage.jsp` 파일을 생성한 후 ② [예제 1-2]의 `Error500.jsp` 코드를 그대로 붙여 넣고 ③ 다음의 [예제 1-4]처럼 상단 지시어 부분에 `errorPage` 속성을 추가
 - 페이지에서 에러가 발생했을 때 직접 처리하지 않고, ①에서 `errorPage` 속성으로 지정한 페이지를 웹 브라우저에 출력
 - 에러 페이지에서는 반드시 `isErrorPage` 속성을 "true"로 설정

1.3 지시어(Directive)(5)

- trimDirectiveWhitespaces 속성
 - 지시어 때문에 생성된 불필요한 공백을 제거하려면 trimDirectiveWhitespaces 속성을 사용
 - page 지시어 속성에 trimDirectiveWhitespaces를 추가하여 true로 설정
- buffer, autoFlush 속성
 - buffer 속성: 응답 결과를 웹 브라우저로 즉시 전송하지 않고, 출력할 내용을 버퍼에 저장했다가 일정량이 되었을 때 전송
 - 버퍼의 크기를 설정(기본값은 8kb)
 - 버퍼를 사용하고 싶지 않다면 "none"으로 지정
 - autoFlush 속성: 버퍼가 모두 채워졌을 때의 처리 방법
 - true(기본값) : 버퍼가 채워지면 자동으로 플러시
 - false : 버퍼가 채워지면 예외를 발생

1.3 지시어(Directive)(6)

1.3.2 include 지시어

- 반복되는 부분을 별도의 파일에 작성해두고 필요한 페이지에서 include 지시어로 포함

```
<%@ include file="포함할 파일의 경로"%>
```

```
<%@ page language="java" contentType="text/html; charset=UTF-8"  
    pageEncoding="UTF-8"%>
```

```
<%@ include file="IncludeFile.jsp" %>
```

—— ① 다른 JSP 파일(IncludeFile.jsp) 포함

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<meta charset="UTF-8">
```

```
<title>include 지시어</title>
```

```
</head>
```

```
<body>
```

```
<%
```

```
out.println("오늘 날짜 : " + today);
```

```
out.println("<br/>");
```

```
out.println("내일 날짜 : " + tomorrow);
```

```
%>
```

```
</body>
```

```
</html>
```

② IncludeFile.jsp에서 선언한 변수 사용

[예제 1-9] 다른 JSP 파일을 포함하는 JSP 파일

1.3 지시어(Directive)(7)

1.3.3 taglib 지시어

- taglib은 EL(표현 언어)에서 자바 클래스의 메서드를 호출하거나 JSTL(JSP 표준 태그 라이브러리)을 사용하기 위한 지시어
 - 자세한 설명은 10장과 11장을 참고

1.4 스크립트 요소(Script Elements)(1)

1.4.1 선언부(Declaration)

- 선언부에서는 스크립틀릿이나 표현식에서 사용할 멤버 변수나 메서드를 선언
- 서블릿으로 변환 시 `_jspService()` 메서드 '외부'에 선언됨

```
<%! 메서드 선언 %>
```

1.4.2 스크립틀릿(Scriptlet)

- JSP 페이지가 요청을 받을 때 실행돼야 할 자바 코드를 작성하는 영역
- 서블릿으로 변환 시 `_jspService()` 메서드 '내부'에 그대로 기술됨
- 선언부에서 정의한 메서드를 호출만 할 수 있을 뿐, 다른 메서드를 선언할 수는 없음

```
<% 자바 코드 %>
```

1.4 스크립트 요소(Script Elements)(2)

1.4.3 표현식(Expression)

- '실행 결과로 하나의 값이 남는 문장'
 - 즉, 상수, 변수, 연산자를 사용한 (수)식, '반환값이 있는' 메서드 호출 등

<%= 자바 표현식 %>

1.4 스크립트 요소(Script Elements)(3)

1.4.4 스크립트 요소 활용

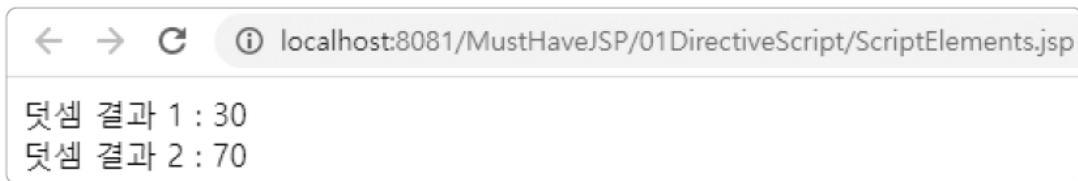
```
<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>
<%!    ① 선언부(메서드 선언)
public int add(int num1 , int num2) {
    return num1 + num2;
}
%>
<html>
<head><title>스크립트 요소</title></head>
<body>
<%    ② 스크립틀릿(자바 코드)
int result = add(10, 20);
%>
덧셈 결과 1 : <%= result %> <br />    ③ 표현식(변수)
덧셈 결과 2 : <%= add(30, 40) %>    ④ 표현식(메서드 호출)
</body>
</html>
```

[예제 1-10] 스크립트 요소 활용

1.4 스크립트 요소(Script Elements)(4)

1.4.4 스크립트 요소 활용

- ① 선언부에서 add()라는 메서드를 정의. 전달받은 두 매개변수의 합을 반환하는 간단한 메서드
- ② 스크립틀릿에서는 add() 메서드에 10과 20을 전달하여 그 결과를 result 변수에 저장
- ③ ④ 표현식을 이용하여 각각 result의 값과 add(30, 40)의 반환값을 출력
 - ④처럼 반환값이 있는 메서드는 표현식에서 바로 호출하는 게 가능



1.4 스크립트 요소(Script Elements)(5)

1.4.4 스크립트 요소 활용

- 실행한 [예제 1-10] 파일이 서블릿으로 변환된 후 컴파일까지 완료되어 생성된 2개의 파일
- ScriptElements.jsp가 처음 실행 시,
 - ScriptElements_jsp.java로 변환된 후
 - ScriptElements_jsp.class로 컴파일
- 메모장으로 .java 파일 열어서 선언부에 정의한 add() 메서드가 _jspService() 외부에 선언된 것을 확인
 - 스크립틀릿에 작성한 코드는 _jspService() 내부에 기술됨
- 스크립틀릿에서 메서드를 선언했을 때 에러가 발생하는 이유
 - 자바에서는 메서드 안에 또 다른 메서드를 선언할 수 없기 때문임

학습 마무리

핵심 요약

- **지시어**
 - **page 지시어** : JSP 페이지에 대한 문서의 타입, 에러 페이지, MIME 타입과 같은 정보 설정
 - **include 지시어** : JSP에서 또 다른 JSP나 HTML 페이지를 포함시킬 때 사용
 - **taglib 지시어** : EL(표현 언어)에서 자바 클래스의 메서드를 호출하거나, JSTL(JSP 표준 태그 라이브러리)을 사용하기 위해 선언 - 10장과 11장에서 학습
- **스크립트 요소**
 - **선언부** : 멤버 변수나 메서드를 선언할 때 사용하는 영역
 - **스크립틀릿** : 선언부에서 선언된 메서드를 호출하거나 자바 코드를 작성하는 영역
 - **표현식** : 주로 변수의 값을 간단하게 출력할 때 사용